CHAPTER 14

EDGE DETECTION

WHAT WILL WE LEARN?

- What is edge detection and why is it so important to computer vision?
- What are the main edge detection techniques and how well do they work?
- How can edge detection be performed in MATLAB?
- What is the Hough Transform and how can it be used to post-process the results of an edge detection algorithm?

What is edge detection?

- Edge detection is a fundamental image processing operation used in many computer vision solutions.
- Goal of edge detection algorithms: to find the most relevant edges in an image or scene.
  - These edges should then be connected into meaningful lines and boundaries, resulting in a segmented image containing two or more regions.
  - Subsequent stages in a machine vision system will use the segmented results for tasks such as object counting, measuring, feature extraction, and classification.

What is edge detection?

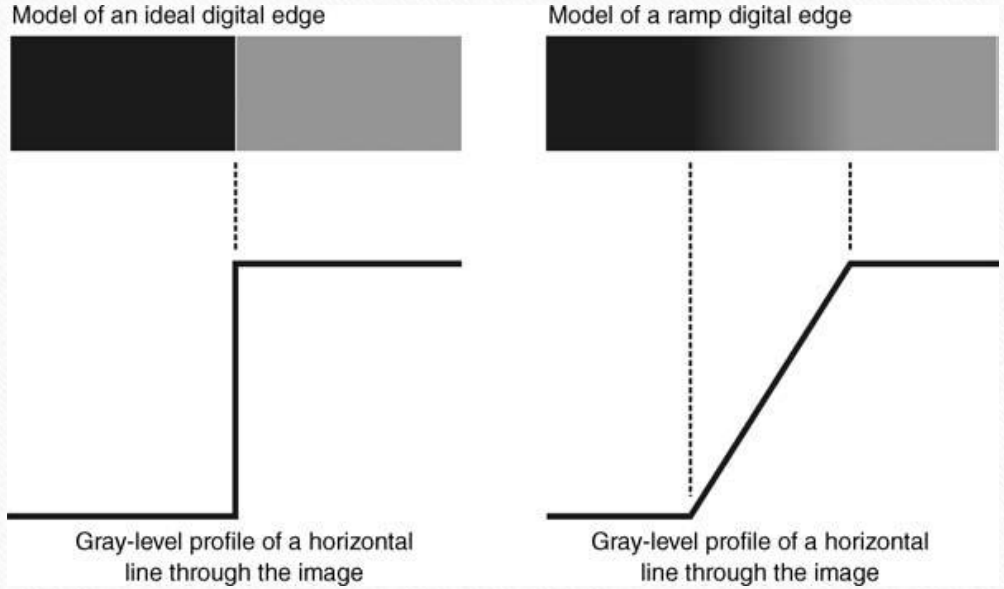
- Biological vision: there is compelling evidence that the very early stages of the human visual system (HVS) contain edge-sensitive cells that respond strongly (i.e., exhibit a higher firing rate) when presented with edges of certain intensity and orientation.
  - Edge detection algorithms attempt to emulate an ability present in the HVS.
  - Such an ability is, according to many vision theorists, essential to all processing steps that take place afterward in the HVS.

What is edge detection?

- Edge detection is a hard image processing problem.
- Most edge detection solutions exhibit limited performance in the presence of images containing real-world scenes.
- It is common to precede the edge detection stage with pre-processing operations such as noise reduction and illumination correction

Basic concepts

- Edge: a boundary between two image regions having distinct characteristics according to some feature (e.g., gray level, color, or texture).
- In grayscale 2D images: a sharp variation of the intensity function across a portion of the image.

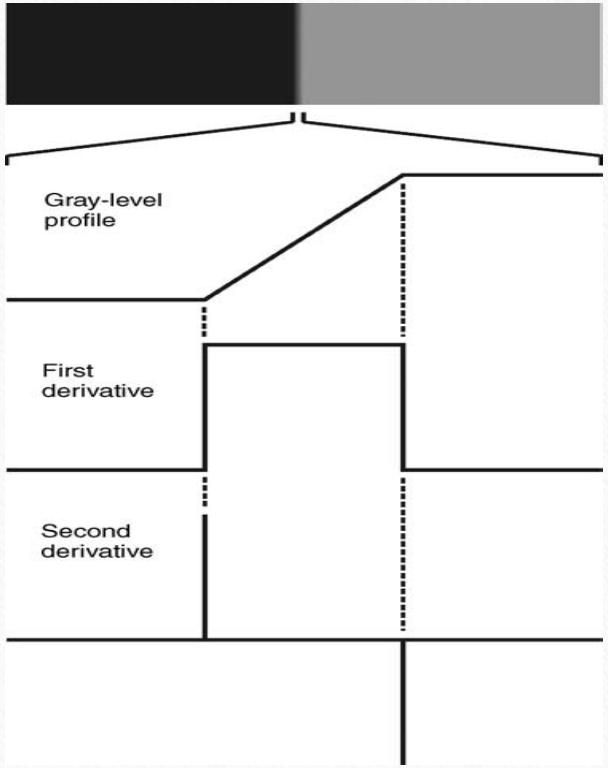


Basic concepts

- Edge detection methods usually rely on calculations of the first or second derivative along the intensity profile.
- The magnitude of the first derivative can be used to detect the presence of an edge at a certain point in the image.
- The sign of second derivative can be used to determine whether a pixel lies on the dark or bright side of an edge.
- Moreover, the zero crossing between its positive and negative peaks can be used to locate the center of thick edges.

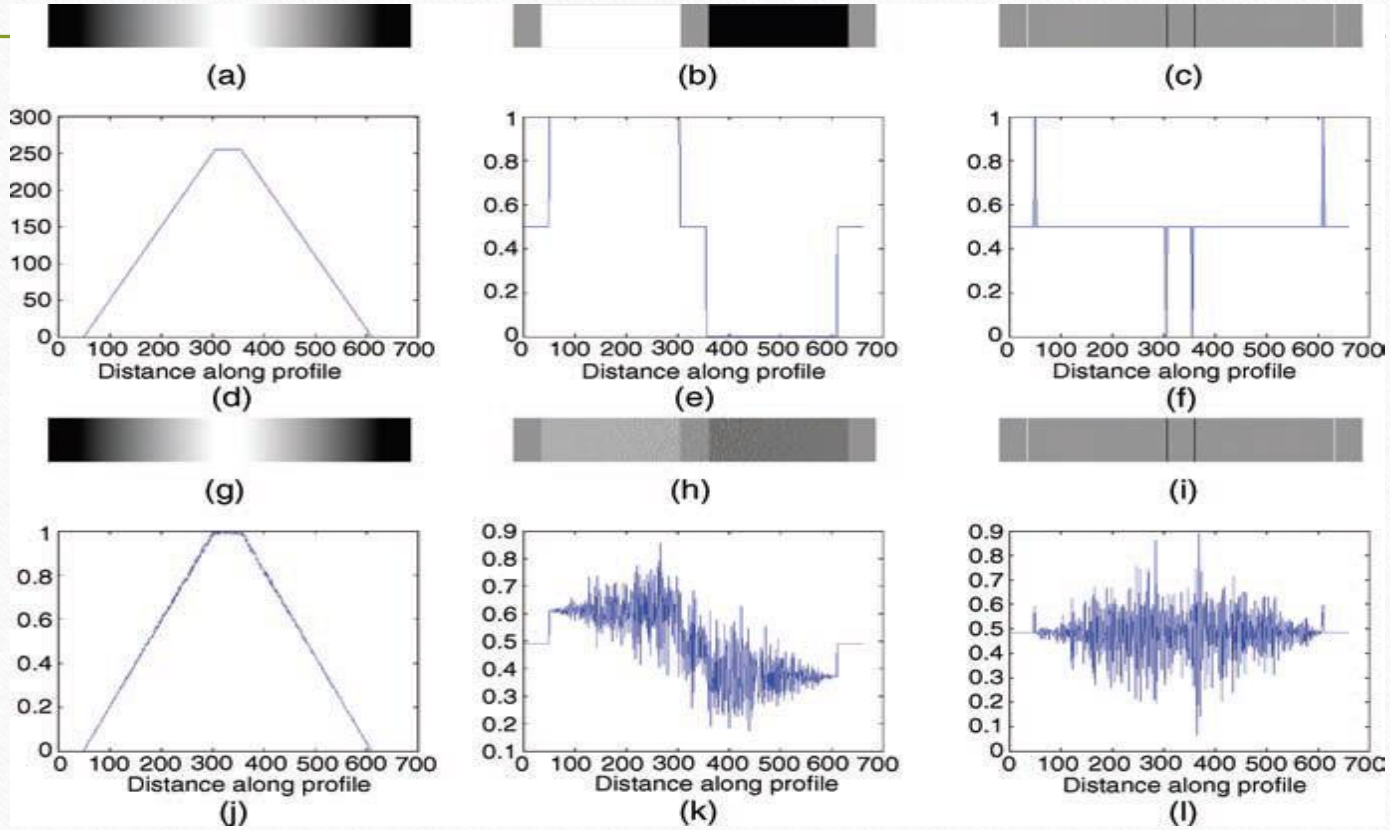
Basic concepts

- First and second derivative



Basic concepts

- The influence of noise



Basic concepts

- The process of edge detection consists of three main steps:
  - Noise reduction
  - Detection of edge points
  - Edge localization
- In MATLAB: **edge**

First-order derivative edge detection

- Digital approximations of the first-order derivative

$$g_x(x,y) \approx f(x+1,y) - f(x-1,y)$$

$$g_y(x,y) \approx f(x,y+1) - f(x,y-1)$$

- Roberts:

$$g_x = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$g_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

First-order derivative edge detection

- Prewitt operators:

$$h_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$h_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

- Sobel operators:

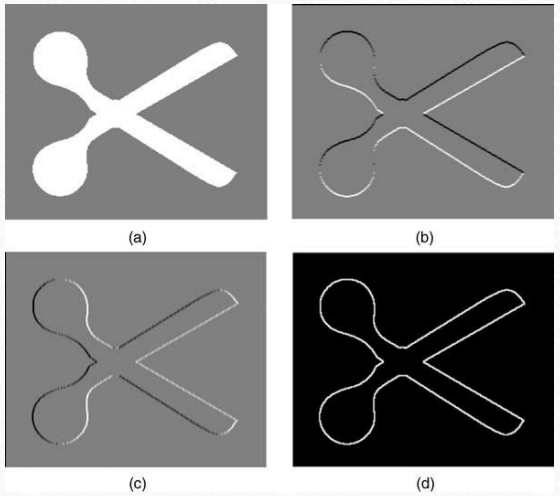
$$h_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$h_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

First-order derivative edge detection

- Prewitt operator

(Example 14.1)

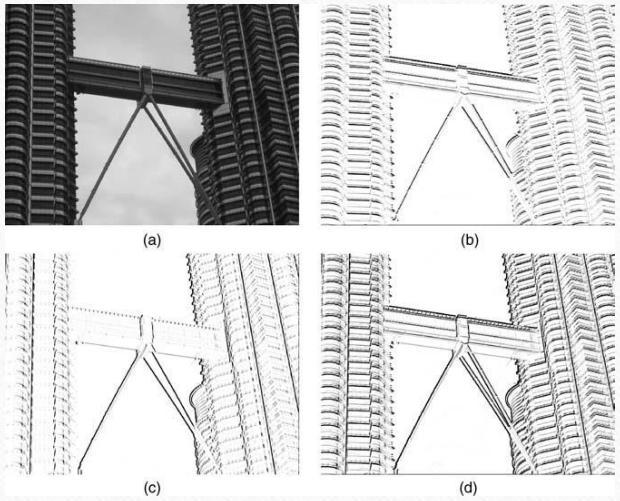



```
J = imread('bw_scissors.png');
Jt = imcomplement(J);
Jt = Jt .* 0.5;
Ju = imcomplement(Jt);
Jd = im2double(Ju);
hy = fspecial('prewitt')
hx = hy'
J8 = imfilter(Jd,hy);
J7 = imfilter(Jd,hx);
Jx = mat2gray(J7);
Jy = mat2gray(J8);
imshow(Jy), figure, imshow(Jx)
J10 = abs(J7) + abs(J8);
Jxy = mat2gray(J10);
figure, imshow(Jd), figure, imshow(Jxy)
```

First-order derivative edge detection

- Sobel operators:

  (Example 14.2)




```
J = imread('Figure14_05_a.png');
Jd = im2double(J);
hy = fspecial('sobel')
hx = hy'
J1 = imfilter(Jd,hy);
J2 = imfilter(Jd,hx);
imshow(J1)
J1c = imcomplement(J1); figure, imshow(J1c)
J2c = imcomplement(J2); figure, imshow(J2c)
J3 = abs(J1) + abs(J2);
J3c = imcomplement(J3); figure, imshow(J3c)
```

First-order derivative edge detection

- Kirsch compass masks:

$k_0 =$

-3	-3	5
-3	0	5
-3	-3	5

$k_1 =$

-3	5	5
-3	0	5
-3	-3	-3

$k_2 =$

5	5	5
-3	0	-3
-3	-3	-3

$k_3 =$

5	5	-3
5	0	-3
-3	-3	-3

$k_4 =$

5	-3	-3
5	0	-3
5	-3	-3

$k_5 =$

-3	-3	-3
5	0	-3
5	5	-3

$k_6 =$

-3	-3	-3
-3	0	-3
5	5	5

$k_7 =$

-3	-3	-3
-3	0	5
-3	5	5

First-order derivative edge detection

- Robinson compass masks:

$r_0 =$

-1	0	1
-2	0	2
-1	0	1

$r_1 =$

0	1	2
-1	0	1
-2	1	0

$r_2 =$

1	2	1
0	0	0
-1	-2	-1

$r_3 =$

2	1	0
1	0	-1
0	-1	-2

$r_4 =$

1	0	-1
2	0	-2
1	0	-1

$r_5 =$

0	-1	-2
1	0	-1
2	1	0

$r_6 =$

-1	-2	-1
0	0	0
1	2	1

$r_7 =$

-2	-1	0
-1	0	1
0	1	2

First-order derivative edge detection

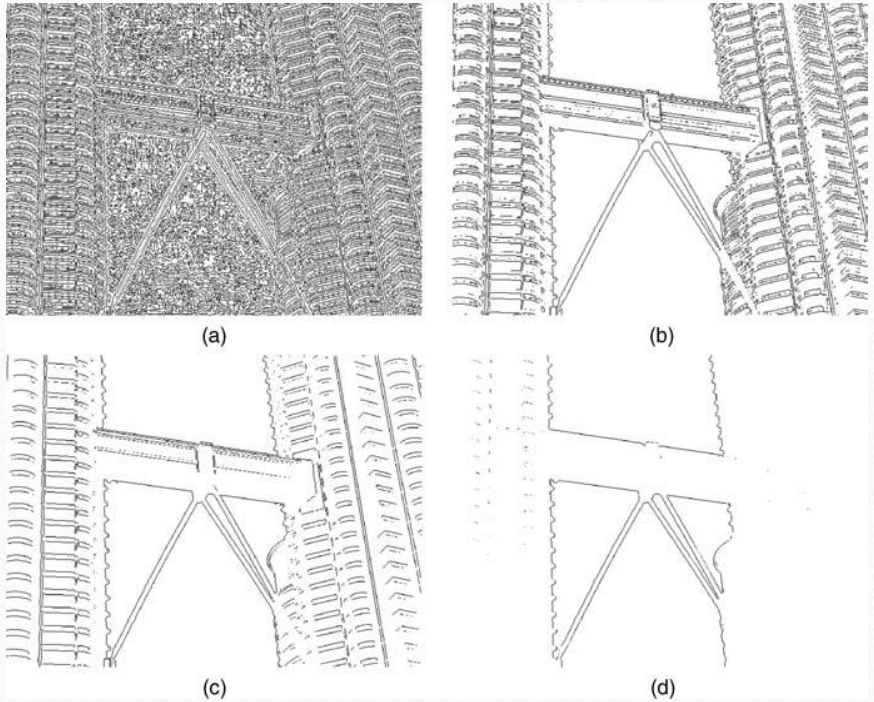
- Sobel operator

with threshold (example)

```
J = imread('Figure14_05_a.png');
thresh = 0;
BW0 = edge(J,'sobel',thresh);
imshow(imcomplement(BW0))
thresh = 0.05;
BW1 = edge(J,'sobel',thresh);
figure, imshow(imcomplement(BW1))
thresh = 0.2;
BW2 = edge(J,'sobel',thresh);
figure, imshow(imcomplement(BW2))
[BW,thresh] = edge(J,'sobel');
thresh
figure, imshow(imcomplement(BW))
```

First-order derivative edge detection

- Sobel operator with threshold (example)



Second-order derivative edge detection

- Problems with Laplacian:
  - It generates “double edges”, i.e., positive and negative values for each edge.
  - It is extremely sensitive to noise.
- In MATLAB:

```
h = fspecial('laplacian',0);
J = edge(I,'zerocross',t,h);
```

Second-order derivative edge detection

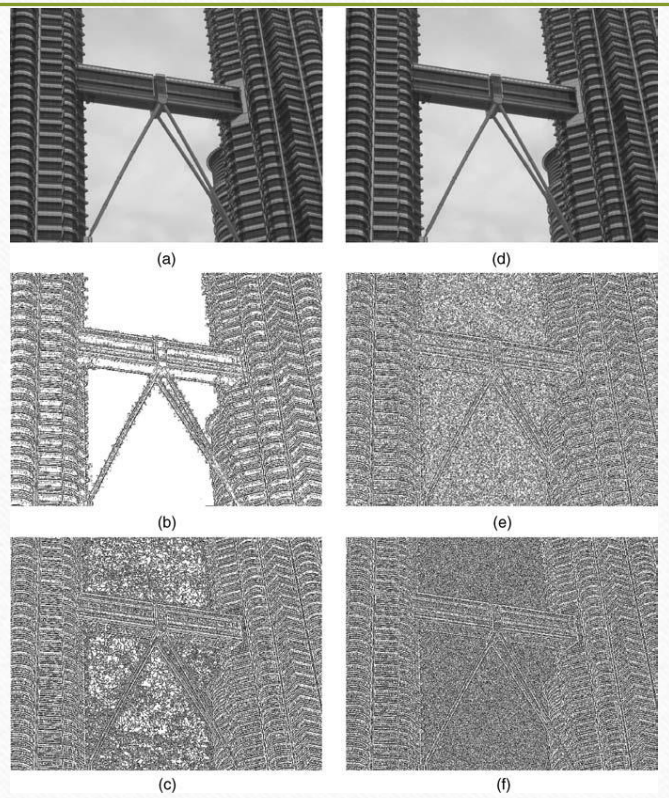
- Laplacian and zero-cross

  (Example 14.3)

```
J = imread('Figure14_05_a.png');
h = fspecial('laplacian',0)
J1 = edge(J,'zerocross',0,h);
J2 = edge(J,'zerocross',[],h);
figure, imshow(imcomplement(J1));
figure, imshow(imcomplement(J2));
K = imnoise(J,'gaussian',0,0.0001);
K1 = edge(K,'zerocross',0,h);
K2 = edge(K,'zerocross',[],h);
figure, imshow(imcomplement(K1));
figure, imshow(imcomplement(K2));
```

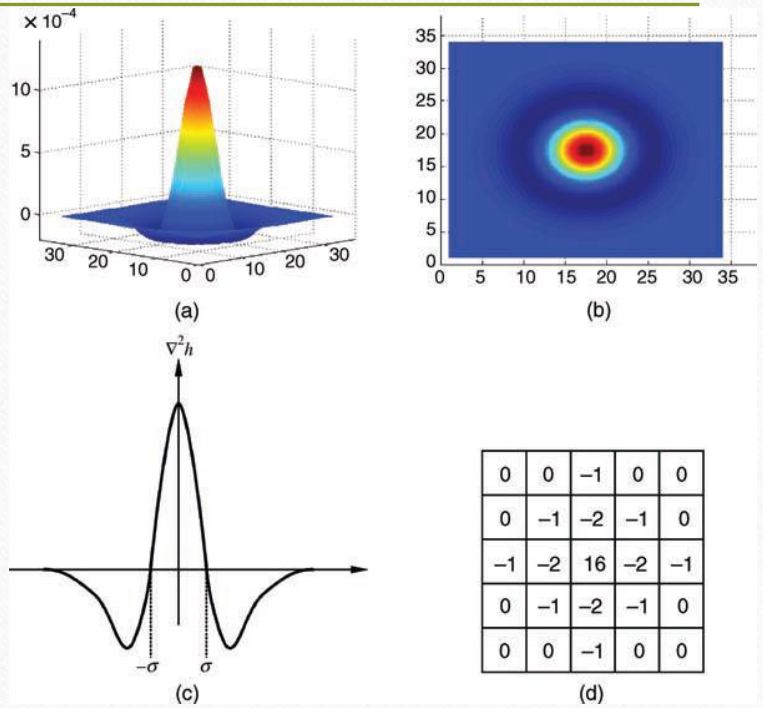
Second-order derivative edge detection

- Laplacian and zero-cross (Example 14.3)



Second-order derivative edge detection

- Laplacian of Gaussian (LoG):
  - Works by smoothing the image with a Gaussian low-pass filter, and then applying a Laplacian edge detector to the result.
  - The LoG filter can sometimes be approximated by taking the differences of two Gaussians of different widths, in a method known as *Difference of Gaussians (DoG)*.

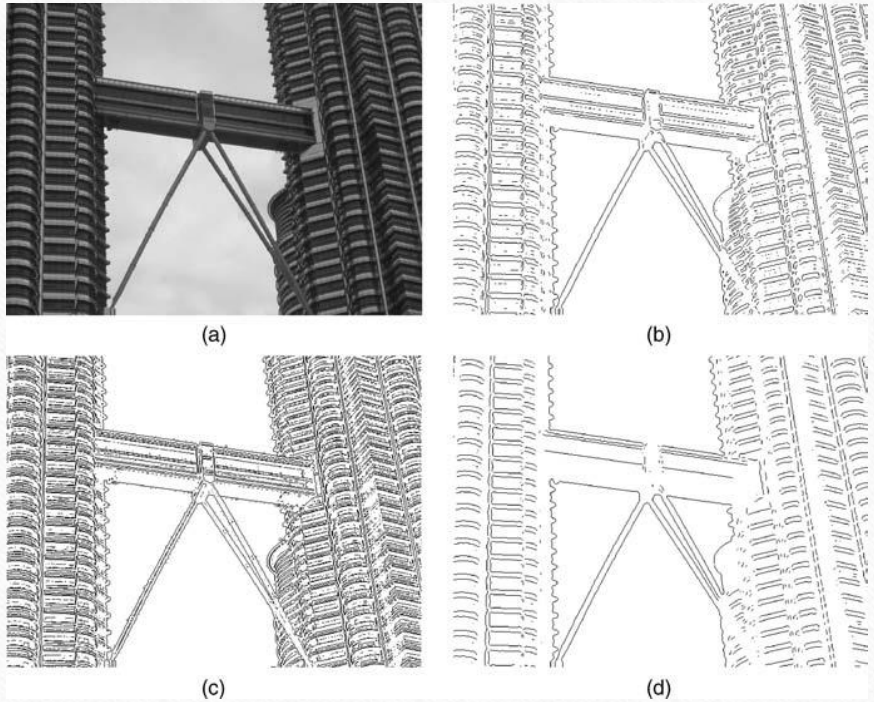


0	0	−1	0	0
0	−1	−2	−1	0
−1	−2	16	−2	−1
0	−1	−2	−1	0
0	0	−1	0	0

Second-order derivative edge detection

- Laplacian of Gaussian (LoG) (Example 14.4):

```
J = imread('Figure14_05_a.png');
J1 = edge(J,'log');
imshow(imcomplement(J1))
J2 = edge(J,'log',[],1);
figure, imshow(imcomplement(J2))
J3 = edge(J,'log',[],3);
figure, imshow(imcomplement(J3))
```



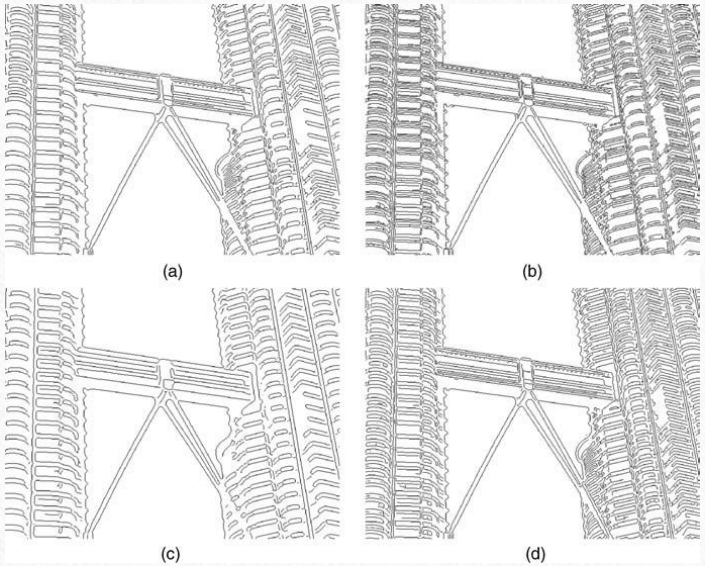
The Canny edge detector

- The input image is smoothed using a Gaussian lowpass filter (LPF), with a specified value of σ.
- The local gradient (intensity and direction) is computed for each point in the smoothed image.
- The edge points at the output of step 2 result in wide ridges. The algorithm thins those ridges, leaving only the pixels at the top of each ridge (*non-maximal suppression*).
- The ridge pixels are then thresholded using two thresholds, Tlow and Thigh: ridge pixels with value greater than Thigh are considered strong edge pixels; ridge pixels with values between Tlow and Thigh are said to be weak pixels. This process is known as *hysteresis thresholding*.
- The algorithm performs edge linking, aggregating weak pixels that are 8-connected to the strong pixels.
- In MATLAB:

  J = edge(I, 'canny', T, sigma);

The Canny edge detector

- Example 14.5:




```
J = imread('twin_towers_skybridge_gray.png');
BW = edge(J,'canny');
[BW, t] = edge(J,'canny');
imwrite(imcomplement(BW),'twin_towers_canny_default.png');
sigma = 0.5
BW = edge(J,'canny',t,sigma);
imwrite(imcomplement(BW),'twin_towers_canny_sigma_half.png');
sigma = 2
BW = edge(J,'canny',t,sigma);
imwrite(imcomplement(BW),'twin_towers_canny_sigma_two.png');
t = [0.01 0.1]
BW = edge(J,'canny',t);
imwrite(imcomplement(BW),'twin_towers_canny_mod_thresh.png');
```

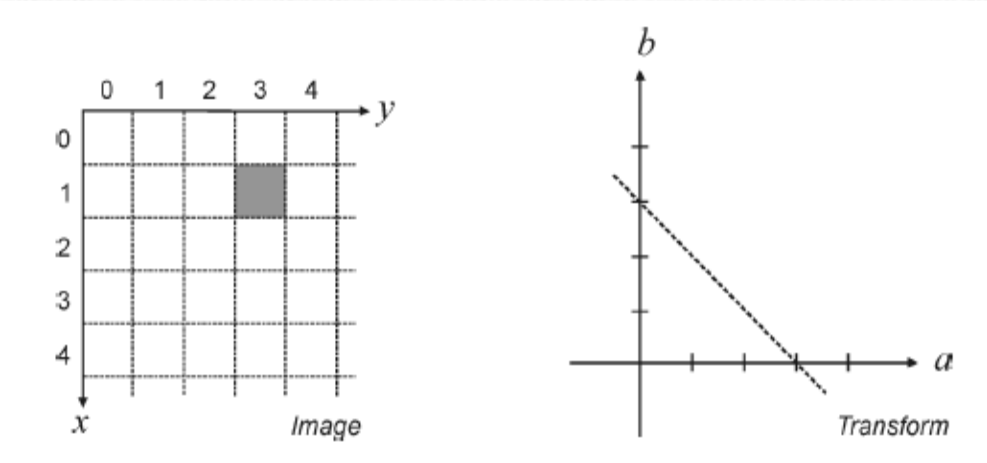
The Hough transform

- A mathematical method designed to find lines in images.
  - It can be used for linking the results of edge detection, turning potentially sparse, broken, or isolated edges into useful lines that correspond to the actual edges in the image.

The Hough transform

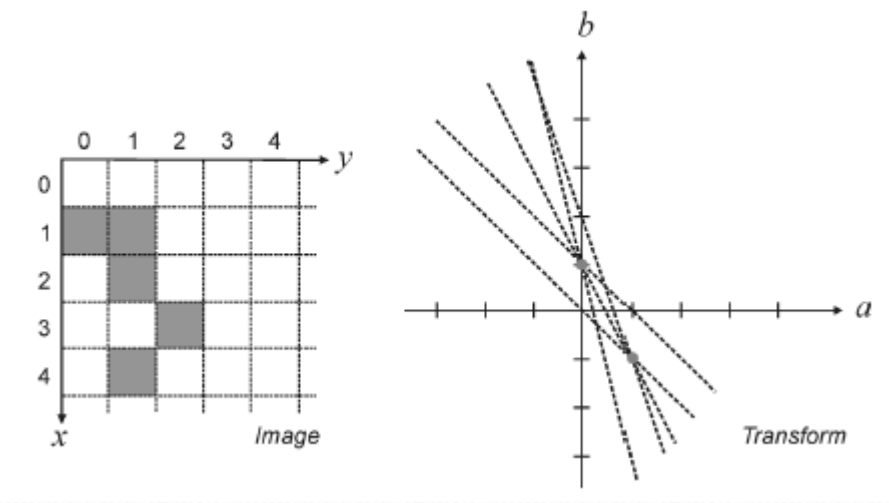
- Let (x,y) be the coordinates of a point in a binary image (containing thresholded edge detection results).
- The Hough transform stores in an accumulator array all pairs (a,b) that satisfy the equation y = ax+ b. The (a,b) array is called the transform array.
  - Example:, the point (x,y) = (1,3) in the input image will result in the equation

    b = -a + 3, which can be plotted as a line that represents all pairs (a,b) that satisfy this equation.



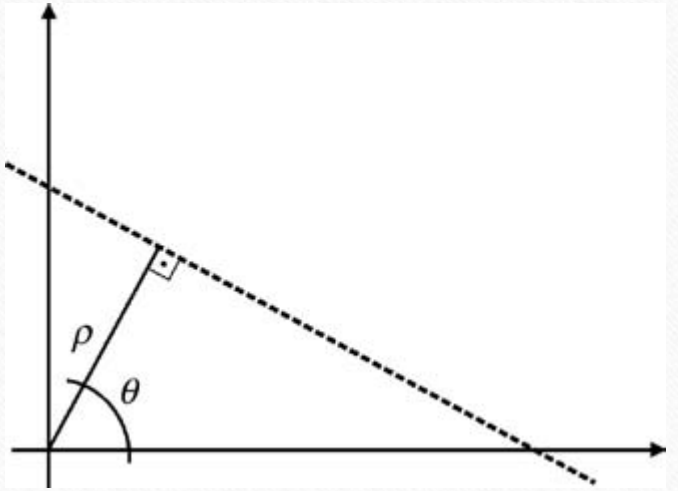
The Hough transform

- Since each point in the image will map to a line in the transform domain, repeating the process for other points will result in many intersecting lines, one per point.
- The meaning of two or more lines intersecting in the transform domain is that the points to which they correspond are aligned in the image.
- The points with the greatest number of intersections in the transform domain correspond to the longest lines in the image.



The Hough transform

- Coordinate conversion
  - Describing lines using the equation $y = ax + b$ (where a represents the gradient) poses a problem, though, since vertical lines have infinite gradient.
    - This limitation can be circumvented by using the normal representation of a line, which consists of two parameters: ρ and θ.
    - In this new representation, vertical lines will have $\theta = 0$.
    - It is common to allow ρ to have negative values, therefore restricting θ to the range $-90° < \theta \leq 90°$.



The Hough transform

- Under the new set of coordinates, the Hough transform can be implemented as follows:
  1. Create a 2D array corresponding to a discrete set of values for ρ and θ. Each element in this array is often referred to as an accumulator cell.
  2. For each pixel (x,y) in the image and for each chosen value of θ, compute $x \cos\theta + y \sin\theta$ and write the result in the corresponding position (ρ, θ) in the accumulator array.
  3. The highest values in the (ρ, θ) array will correspond to the most relevant lines in the image.

The Hough transform

- In MATLAB:
  - The IPT contains a function for Hough transform calculations, hough, which takes a binary image as an input parameter, and returns the corresponding Hough transform matrix and the arrays of ρ and θ values over which the Hough transform was calculated.
  - Optionally, the resolution of the discretized 2D array for both ρ and θ can be specified as additional parameters.

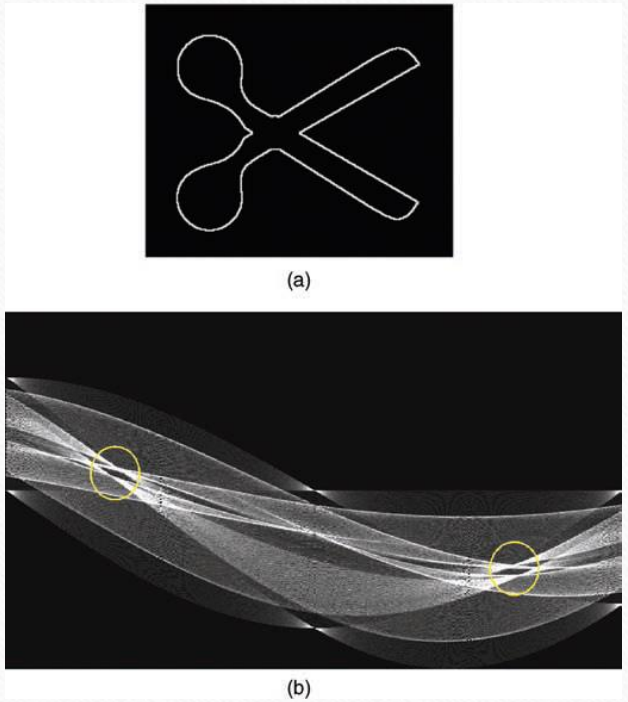
The Hough transform

- Example 14.6

```
BW = imread('Figure14_16_a.png');
imshow(BW)

[H,T,R] = ...
      hough(BW,'RhoResolution',0.5,'ThetaResolution',0.5);

figure, imshow(imadjust(mat2gray(H)),'XData', ...
      T,'YData',R, 'InitialMagnification','fit');

xlabel('\theta'), ylabel('\rho');
axis on, axis normal, hold on;
colormap(gray);
```

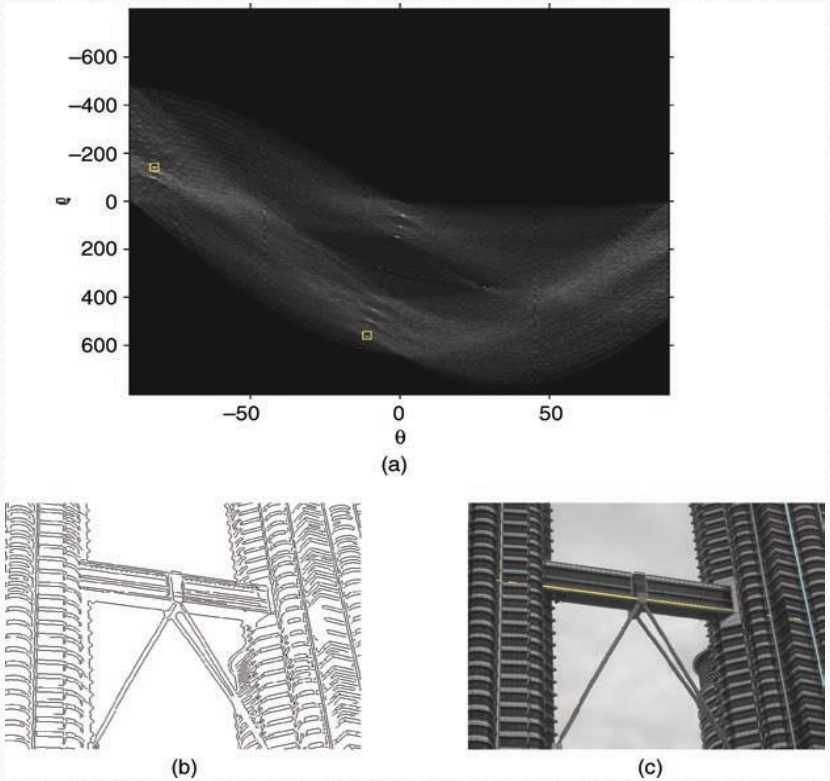
The Hough transform

- In MATLAB:
- The IPT also includes two useful companion functions for exploring and plotting the results of Hough Transform calculations:
- **houghpeaks:** identifies the k most salient peaks in the Hough transform results, where k is passed as a parameter.
- **houghlines:** draws the lines associated with the highest peaks on top of the original image.



The Hough transform

- Example 14.7



Hands-on

- Tutorial 14.1: Edge detection (page 354)